

LOGISTIC MANAGEMENT SYSTEM SEMAPHORES AND MUTEXES

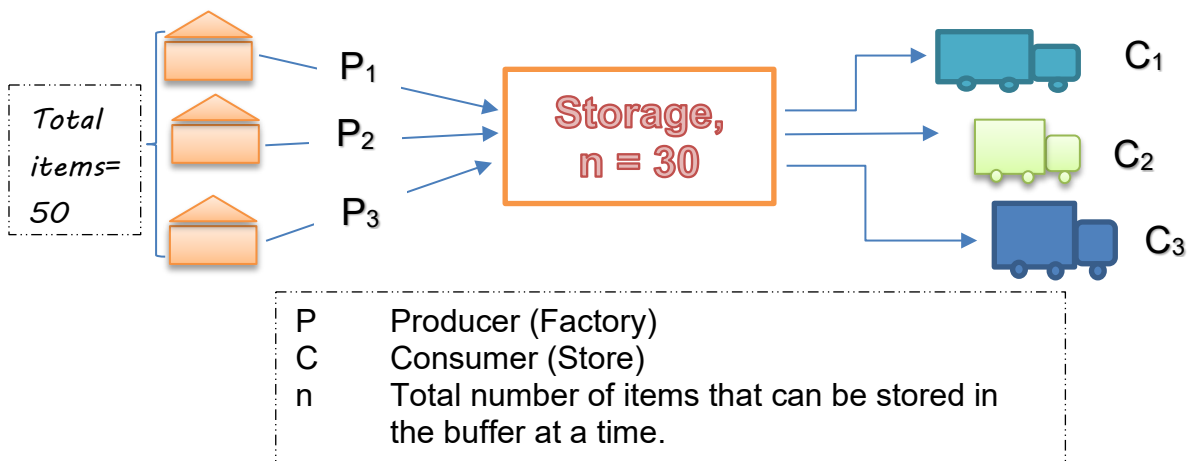
1 OBJECTIVES

The main goals of this assignment are to:

- To gain knowledge about semaphores and mutexes as synchronization mechanisms.
- To run multiple threads concurrently using semaphores and mutexes.

2 DESCRIPTION

This assignment simulates a transportation system connecting product manufacturers (producers) and retail companies (consumers). Independent factories generate items and deliver them to a shared storage facility, while retailers (e.g., grocery stores) retrieve these items from the storage. The storage has a fixed capacity, enforcing a maximum number of items that can be stored at once. For simplicity, the simulation restricts the number of producers and consumers to three each.



The illustration above depicts the scenario for this assignment, modeling a logistics system with producer factories, a shared warehouse, and consumer factories. The **producers** (factories) manufacture items and use trucks to deliver them to the buffer—but only if space is available. Conversely, the **consumers** (trucks) retrieve items from the warehouse—but only if it is not empty.

The storage object (shared buffer) acts as a coordinator between producer and consumer threads. It provides a fixed-capacity buffer (implemented as an array) where producers store items and consumers retrieve them. Since the buffer has limited space, the storage object must synchronize access to ensure thread-safe operations.

3 INSTRUCTIONS AND REQUIREMENTS:

Develop a multithreaded application simulating the logistics scenario described earlier. The application must meet the following requirements.

a. GUI Implementation

- Include a graphical interface to clearly visualize concurrent thread operations, making it easier to observe thread behavior.
- Ensure thread interactions are free from interference and race conditions while maintaining safety and liveness properties.
- The assignment page on Canvas provides GUI code for both Java and C#. You may use the provided design or modify/create your own GUI layout.

b. Synchronization Mechanisms (semaphores mandatory)

- Coordinate shared data access using both semaphores and mutexes.
- Secure critical sections with a mutex (binary semaphore in Java)
- Implement proper semaphore signaling to prevent Deadlock, Starvation and other unintended behavior
- Strictly follow the fundamental producer-consumer rules:
 - Producers must wait when buffer is full.
 - Consumers must wait when buffer is empty.
 - Only one thread may access the buffer at any time.

c. Object Representation

Model manufacturers as Producer (or Factory) objects, retailers as Consumer (or Store) objects, and storage as a Storage (or SharedBuffer) class.

Producer

- Implement a Producer class that maintains access to an array of Product objects for delivery to the Storage (shared buffer). It delivers products one item at a time in a loop

Consumer

- Implement a Consumer (or Store) class. It should not access the Producer's product array or the total number of products.
- It however may maintain a separate array to store consumed products.

Storage

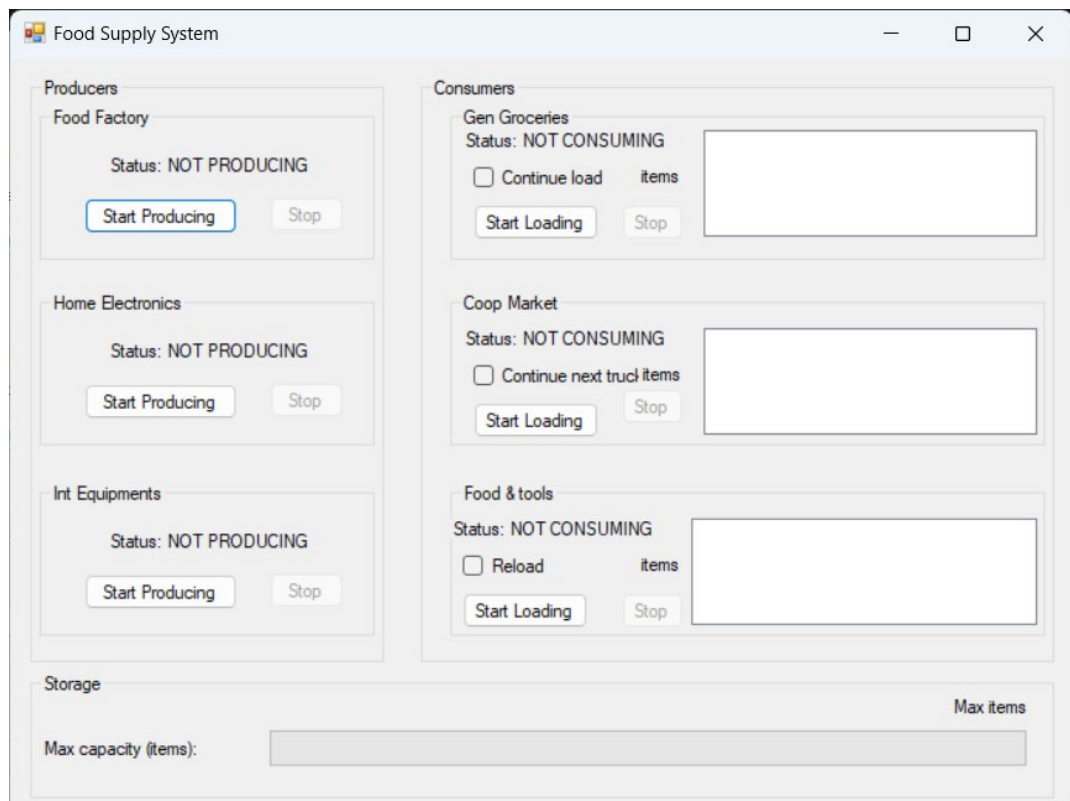
- Implement a Storage (or SharedBuffer) class.
- The Storage should have a buffer with a limited capacity other than the number of source items. Use a fixed-size array of Product objects as **bounded** buffer.

```
const/final int buffersize = 30;
Products [ ] buffer = new Product[buffersize];
```

- Do not use dynamic collections (e.g. List).
- The class should not access the Producer's product array or the total number of products.
- The class must provide the following methods in addition to other necessary ones:
 - **Produce**(Product item): Called by producer threads to add items.
 - **Consume**(): Called by consumer threads to remove items. It may return the consumed product.

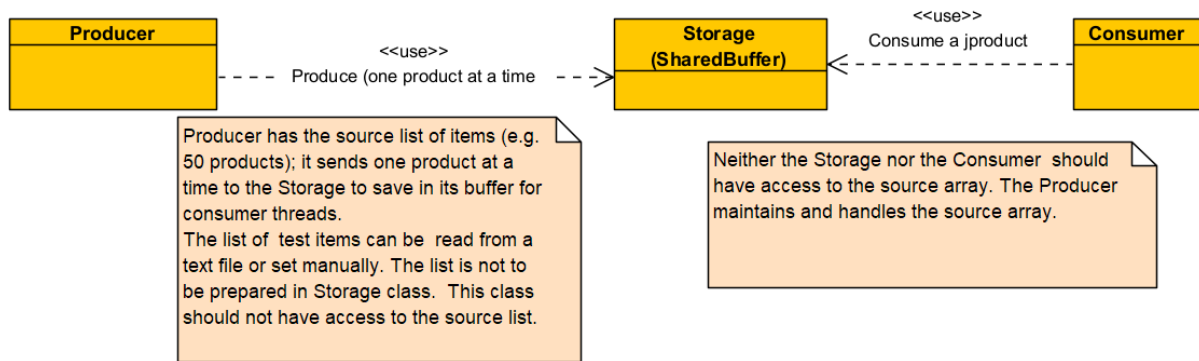
d. GUI Design

A suggested GUI design is shown below (with complete C# and Java code available on the assignment page. Producer/consumer naming is flexible (e.g., "Acme Factory", "Global Mart").



4 SOLUTION PATTERN AND OTHER REQUIREMENTS

4.1 Follow the pattern below when designing your solution.:



4.2 When a consumer truck reaches full capacity, the user has two options:

- Stop and restart the thread, or
- Select "Continue load" option

If the "Continue load" option is enabled, the consumer truck waits for a configured time interval, and then:

- clears its current item list, and
- resumes loading automatically.
- If the buffer is empty, the **consumers** will wait until items become available.
- If the buffer is full, **producers** will wait until the consumers remove items.

Each consumer thread (truck) should have a maximum capacity of items :

Consumer 1: Maximum 12 items

Consumer 2: Maximum 15 items

Consumer 3: Maximum 10 item

4.3 There is no maximum limit for each producer thread. However, the total number of items produced by all three threads combined should be limited to a certain number, for example, 50.

5 GENERAL REQUIREMENTS

5.1 The GUI should display relevant information such as the current status of the threads. Update the status labels on the GUI (refer to the GUI image presented earlier) for both the

producer and consumer threads to reflect their actual status, such as "Not consuming," "Producing," or "Waiting."

- 5.2 Create a minimum of three producer threads and three consumer threads, with the option to experiment with more threads if desired.
- 5.3 Prepare a test list of product items and make it available to the Producer class. The list can be prepared inside the Producer class as well. It can be copied from an online source, created manually, or loaded from a text file.
- 5.4 Each producer thread should randomly pick an arbitrary item from the collection and place it in the buffer, with a time interval between each production.
- 5.5 You may certainly make changes to the application to make it more advanced and full-featured.

6 THE PRODUCT CLASS:

In addition to GUI and other standard classes such as the start class (Main or Program), as well as the classes described earlier, you may need to create a class to represent a Product item:

6.1 The product class

Create a class product with attributes: name, price and a category type:

```
class Product
{
    • string name
    • double price
    • Category (Food, Electronic or Tool)
```

```
enum CategoryType
{
    Food,
    Electrics,
    Tools
}
```

Implement a ToString() method in C# or a toString() method in Java to return a string in the format "name - price" for display on the GUI.

7 SUBMISSION AND GENERAL REQUIREMENTS FOR A PASSING GRADE (G)

To qualify for a pass grade, you should maintain good code quality and a well-organized project. Don't forget to document your code by writing comments.

Build and test your application carefully before submitting; make sure there are no bugs.

Submit the assignment as previously. Upload your solution as a compressed file either before or after showing it to your lab supervisor.

Good Luck!

Farid Naisan,
Course Responsible and Instructor